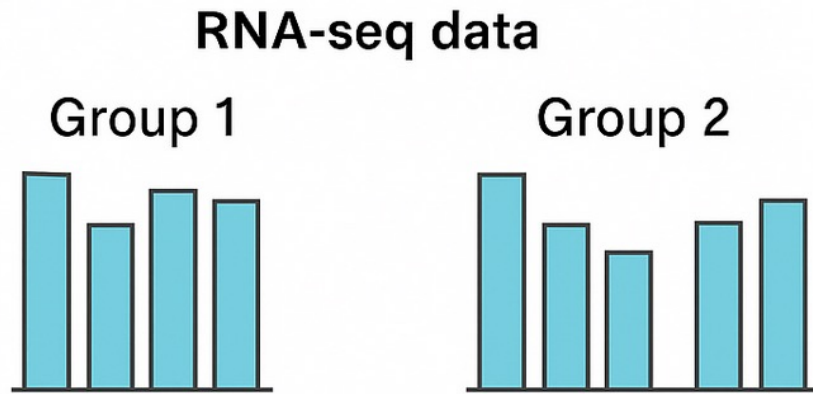
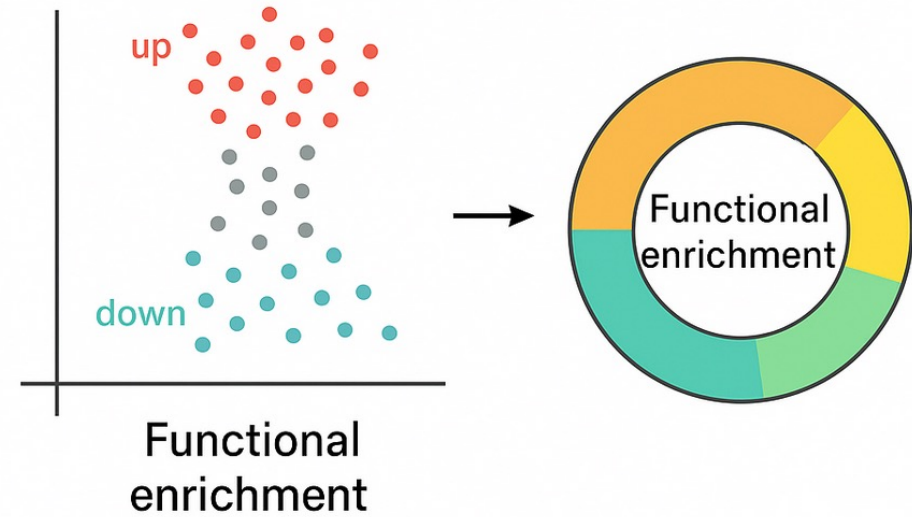


RNA-seq Analysis

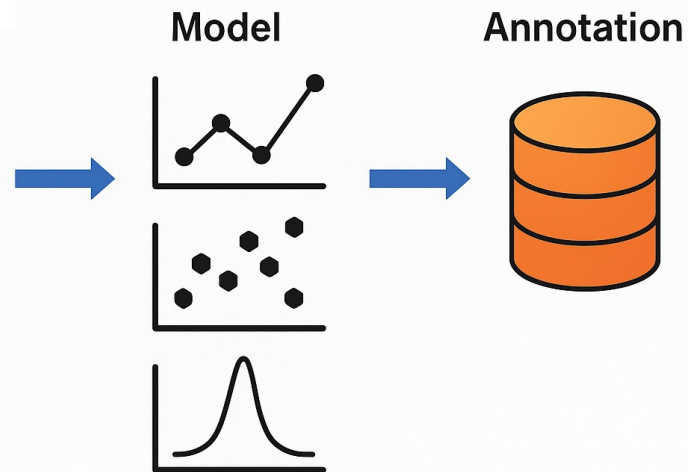


Differentially expressed genes



Sample / Group / Replication

Sample 1	4	19
Sample 2	11	26
Sample 3	12	32
...	6	10
Sample 4	10	28
Sample 5	14	39
Sample 6	16	22



How can I obtain this raw data?

Standardized data

GeneA	Sample2	Sample3
GeneA	1.2	3.2
GeneB	0.2	1.8
GeneC	0.0	2.1

Input FASTQ files



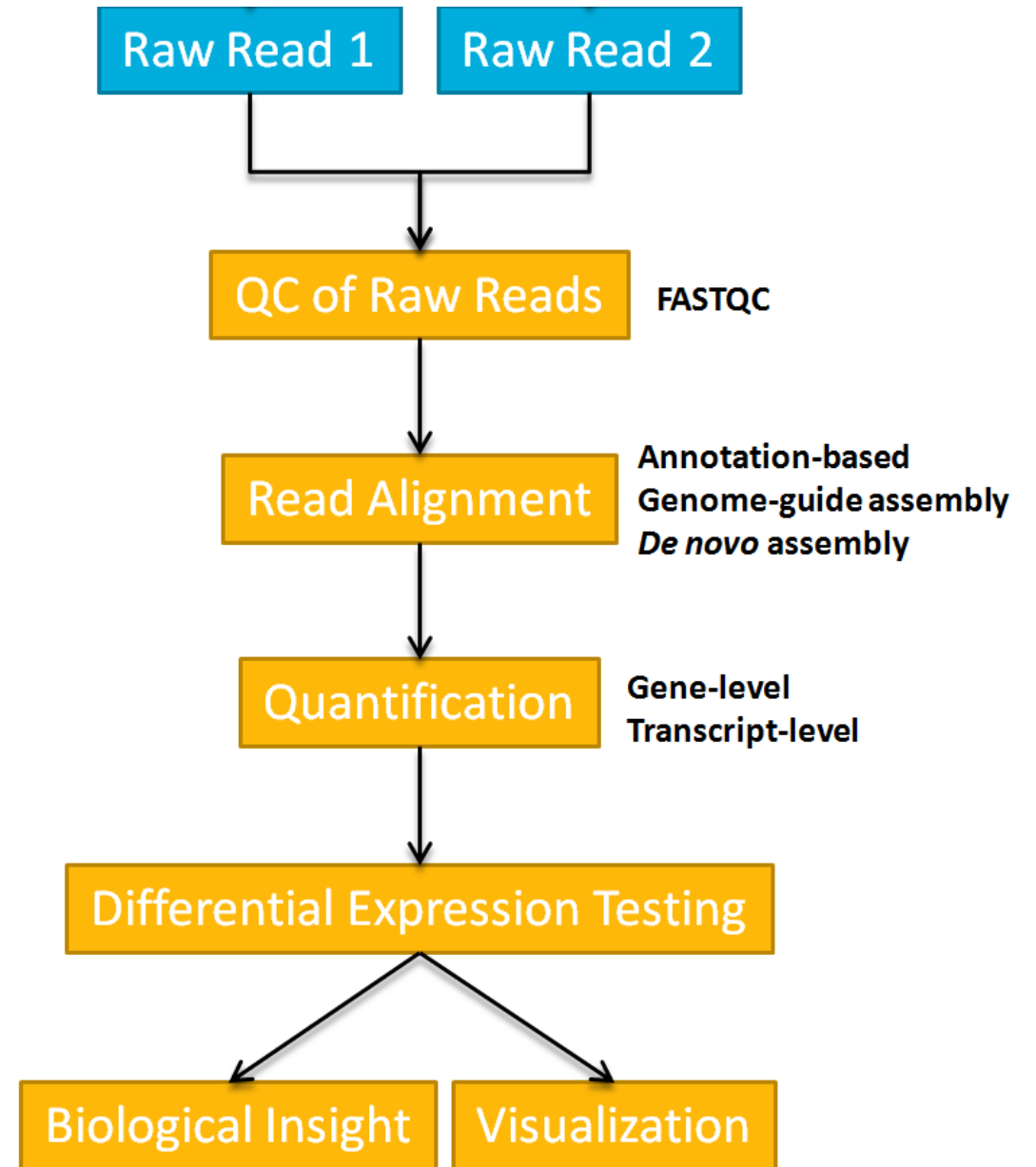
Input FASTQ files



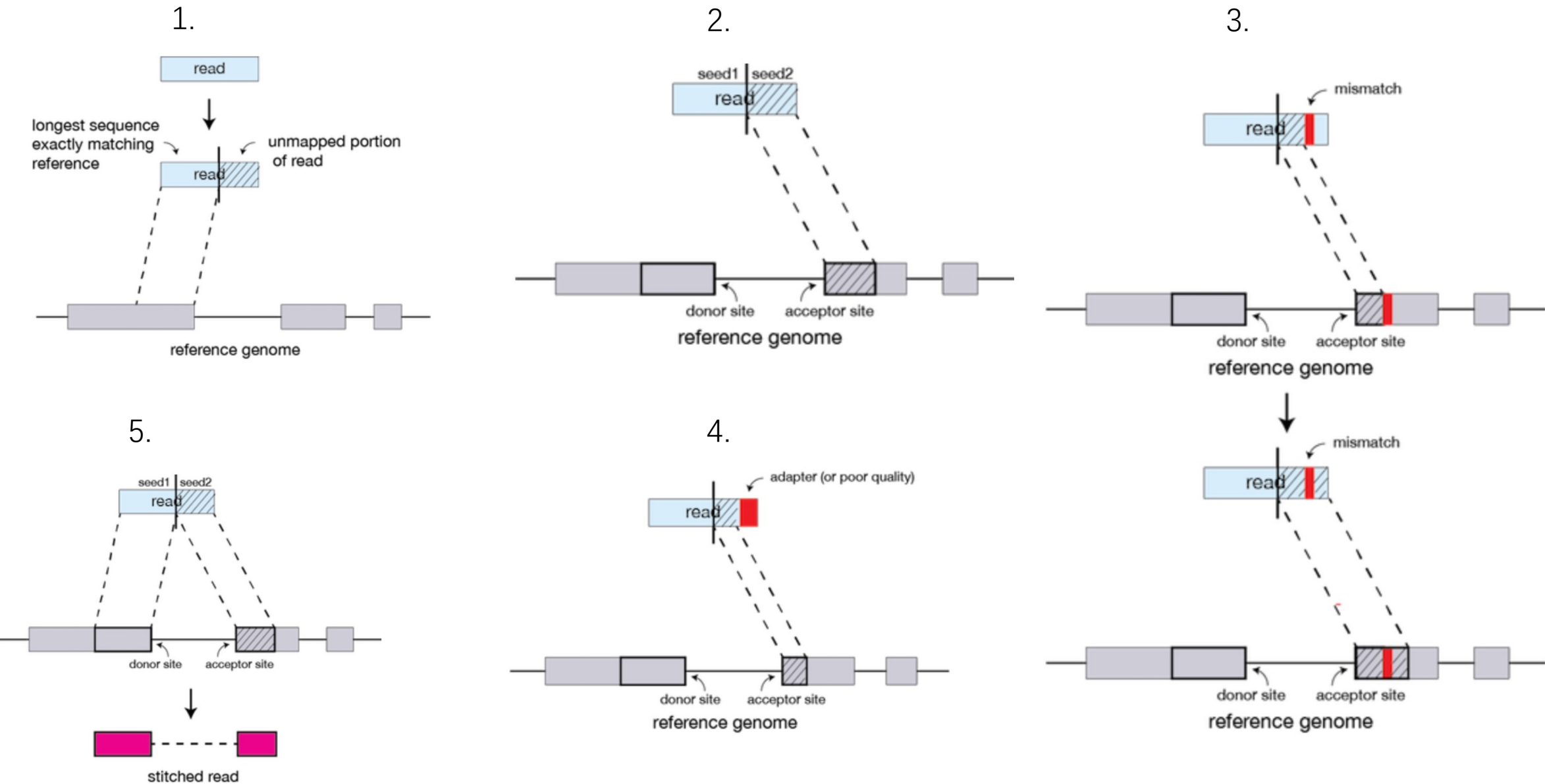
How can I obtain this raw data?

Standardized data

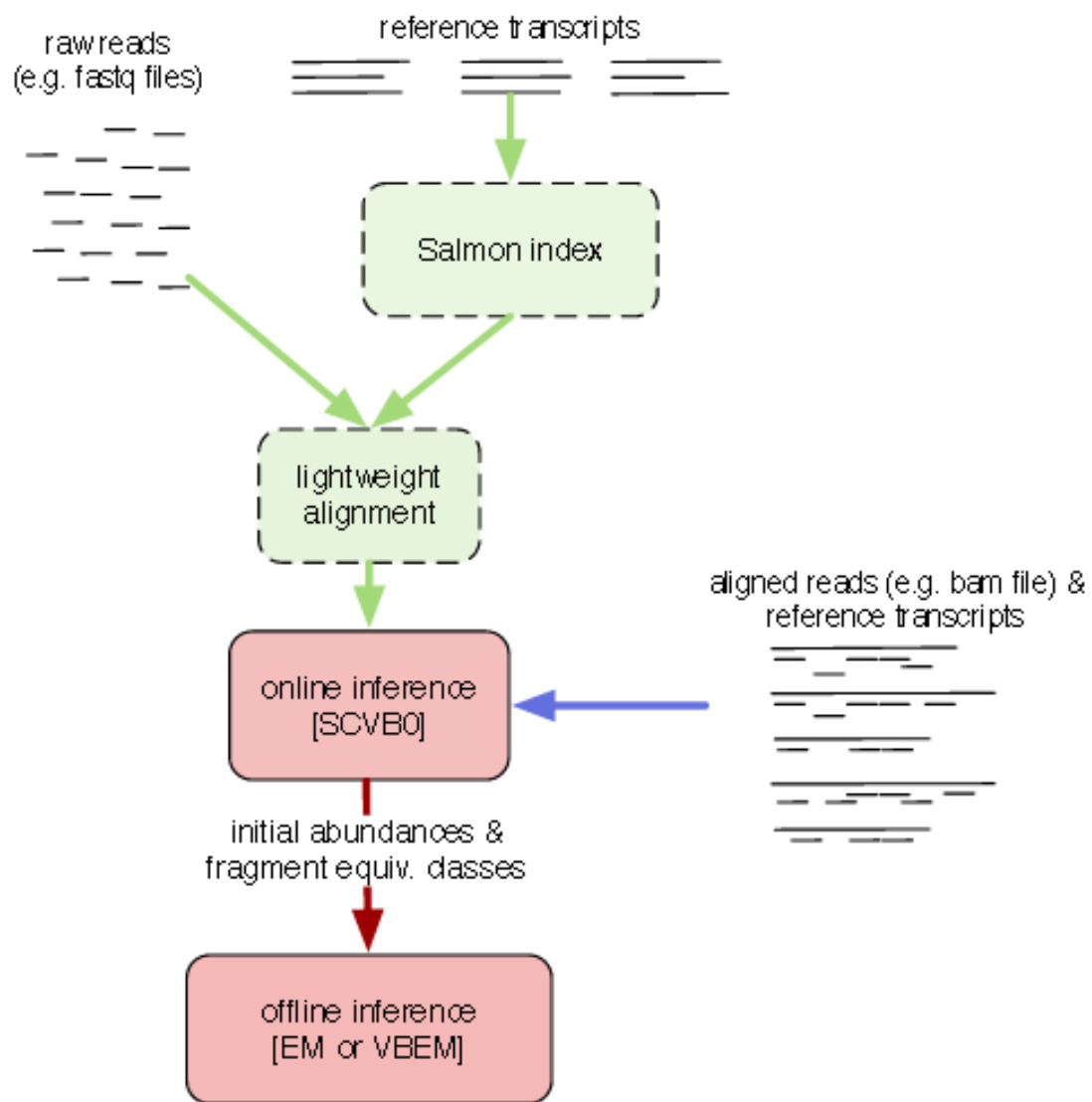
GeneA	Sample2	Sample3
GeneA	1.2	3.2
GeneB	0.2	1.8
GeneC	0.0	2.1



RNA-Seq Mapping with STAR



Salmon – Accurate, Versatile and Ultrafast Quantification from RNA-seq Data using Lightweight-Alignment



特点:

- 非常快 (几小时的 STAR 对齐, Salmon 十几分钟搞定)。
- 不会产生 BAM 文件 (下游TBCR分析的时候需要)

PYTHON / R

机器语言

B8 02 00 00 00

83 C0 03

50

E8 xx xx xx xx

汇编语言

mov eax, 2 ; 把 2 放到寄存器 eax

add eax, 3 ; eax = eax + 3

push eax ; 把结果压栈, 作为参数传给 printf

call printf ; 调用打印函数

高级语言

a = 2

b = 3

print(a + b)



R语言基础语法

1. 基本数据类型

数值型 (numeric) : 用于存储整数和小数。

整数型 (integer) : 整数, 定义时在数字后加L, 如5L。

字符型 (character) : 字符串, 如"Hello, R"。

逻辑型 (logical) : 布尔值, TRUE或FALSE。

复数型 (complex) : 复数, 如2+3i。

2. 数据结构

向量 (Vector) :

R中最基本的数据结构, 包含相同类型的元素。

创建向量:

```
# 数值向量
```

```
num_vec <- c(1, 2, 3, 4, 5)
```

```
# 字符向量
```

```
char_vec <- c("a", "b", "c")
```

```
# 逻辑向量
```

```
log_vec <- c(TRUE, FALSE, TRUE)
```

矩阵 (Matrix)

- 二维的数据结构, 元素类型相同。

- 创建矩阵:

- # 创建一个2行3列的矩阵

- `mat <- matrix(1:6, nrow = 2, ncol = 3)`

数组 (Array)

- 多维的数据结构, 元素类型相同。

- 创建数组:

2. 数据结构

列表（List）：

可以包含不同类型和不同长度的元素。

创建列表：

```
lst <- list(  
  name = "Alice",  
  age = 25,  
  scores = c(85, 90, 95)  
)
```

变量与赋值

变量名区分大小写，通常使用<-进行赋值。

```
x <- 10  
y <- "Hello"
```

数据框（Data Frame）：

类似于数据库中的表，每列可以是不同的数据类型。

创建数据框：

```
df <- data.frame(  
  ID = 1:3,  
  Name = c("Alice", "Bob", "Charlie"),  
  Score = c(85, 92, 78)  
)
```

运算符

- **算术运算符**：+、-、*、/、^（幂）、%%（取模）、%/（整数除法）。
- **比较运算符**：>、<、>=、<=、==、!=。
- **逻辑运算符**：&（且）、|（或）、!（非）。

3、基本语法

•if-else 语句

•if (condition) { # 当条件为真时执行 } else { # 当条件为假时执行 }

•ifelse 函数

•result <- ifelse(condition, value_if_true, value_if_false)

示例：

```
score <- 85
if (score >= 90) { grade <- "A" }
else if (score >= 80) { grade <- "B" } else { grade <- "C" }
```

循环语句

•for 循环

for (variable in sequence) { # 循环体 }

•while 循环

while (condition) { # 循环体 }

示例：

```
# 打印1到5
for (i in 1:5) { print(i) }
```

四、函数和包管理

定义函数

```
my_function <- function(arg1, arg2) {
  # 函数体
  result <- arg1 + arg2
  return(result)
}
```

调用函数

```
sum <- my_function(5, 3) # sum = 8
```

常用内置函数

数学函数：abs()、sqrt()、log()、exp()、sin()、cos()。

统计函数：mean()、median()、sd()、var()、summary()。

字符函数：nchar()、substr()、grep()、gsub()。

5. 包的安装和加载

安装包

```
install.packages("包名")
```

加载包

```
library(包名)
```

示例

```
install.packages("ggplot2")  
library(ggplot2)
```

如果没有包，如何实现画一个点图？

```
function plot(x, y):  
    // 1. 准备数据  
    check_input(x, y)           // 确认x,y长度一致，类型正确  
    n <- length(x)  
  
    // 2. 打开图形设备（屏幕、PDF、PNG…）  
    device = open_graphics_device("X11") // Linux 上可能是 X11  
                                     // Windows 上可能是 winGD  
  
    // 3. 分配内存缓冲区  
    buffer = malloc(width * height * 4) // RGBA 缓冲区，用来存像素点  
  
    // 4. 设置绘图参数  
    set_margins(buffer, top, bottom, left, right)  
    set_axis_limits(buffer, min(x), max(x), min(y), max(y))
```

```
// 5. 绘制坐标轴和网格  
draw_axis(buffer, "x")  
draw_axis(buffer, "y")  
draw_grid(buffer, …)  
  
// 6. 把点映射到像素坐标  
for i in 1..n:  
    px = map_to_pixel(x[i], axis_x)  
    py = map_to_pixel(y[i], axis_y)  
    draw_point(buffer, px, py, color="black", size=3)  
  
// 7. 如果有标题/图例  
draw_text(buffer, position, "title", font="Helvetica")  
draw_legend(buffer, …)  
  
// 8. 把缓冲区内容刷新到屏幕  
flush(buffer, device)  
  
// 9. 释放资源  
free(buffer)  
close(device)
```

有了包以后：
ggplot(…)

edgeR

输入： raw counts 矩阵（基因 × 样本）。

构建 DGEList：

过滤低表达基因（提高统计功效）。

标准化 (normalization)： TMM (trimmed mean of M values) 。

估计离散度：

common dispersion（全局）

trended dispersion（随表达量变化）

tagwise dispersion（每个基因单独）

建模与检验：

类似广义线性模型 (GLM)

exactTest 或 glmLRT 得到差异基因。

结果输出： logFC、p-value、FDR。

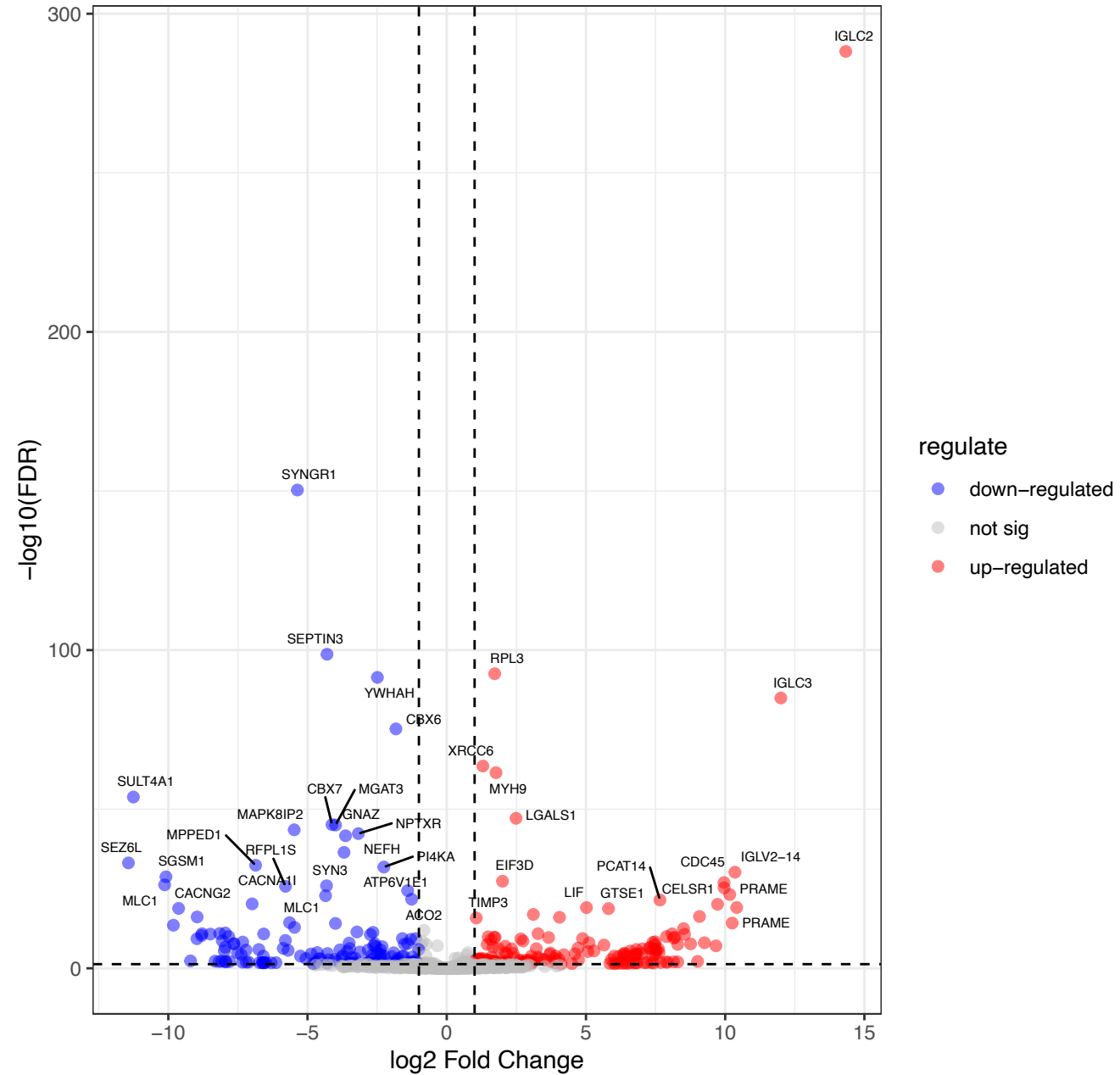
$$\log(\mu_i) = \beta_0 + \beta_1 \cdot \text{condition}_i$$

- μ_i ：样本 i 在该基因上的期望表达（归一化后）
- β_0 ：截距 → **HBR 的平均表达**
- β_1 ：conditionUHR 的系数 → **UHR 相对 HBR 的差异 (log-fold change)**

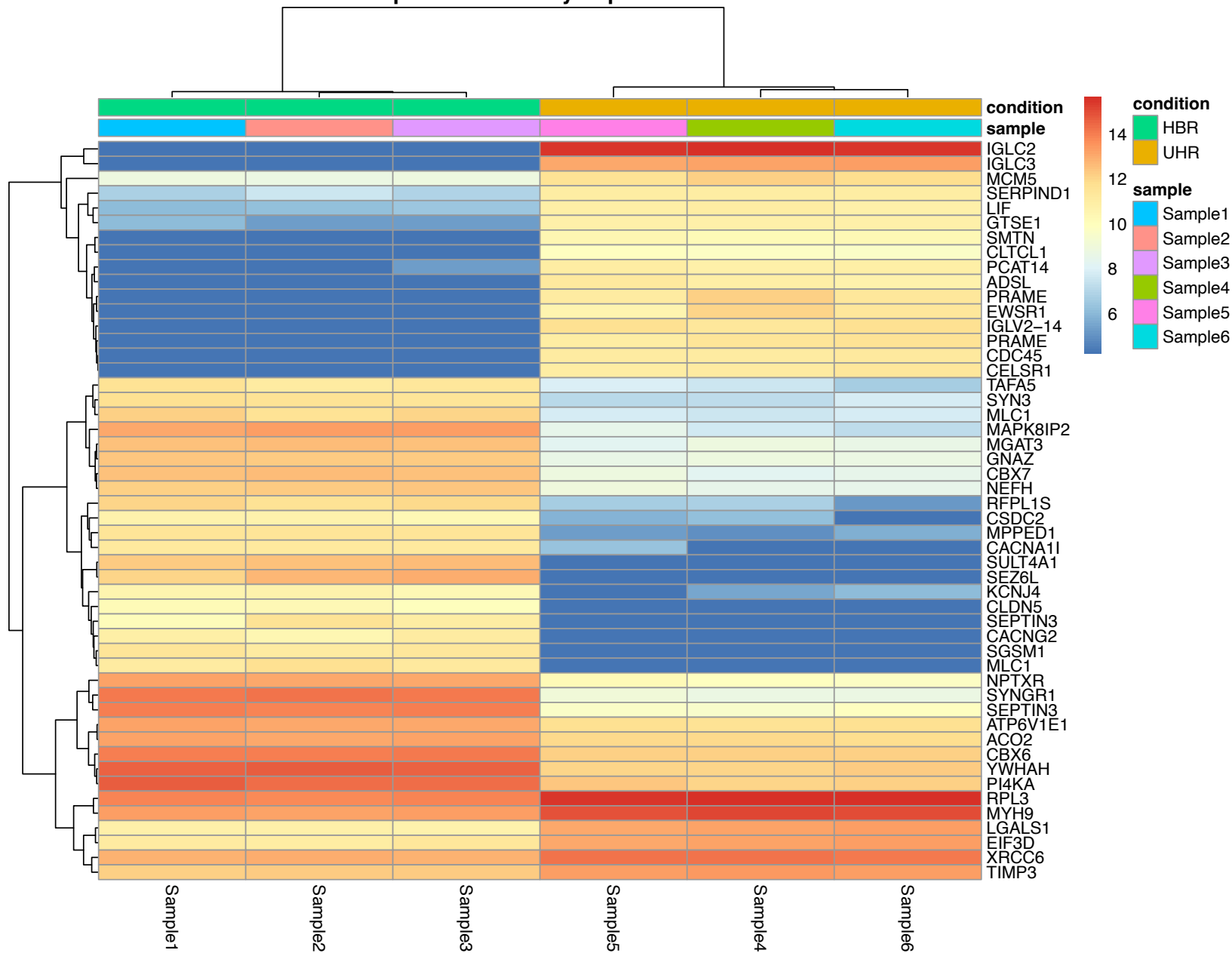
$$\text{Var}(Y_i) = \mu_i + \phi \mu_i^2$$

logFC / 它的标准差 -> p-value

Volcano Plot

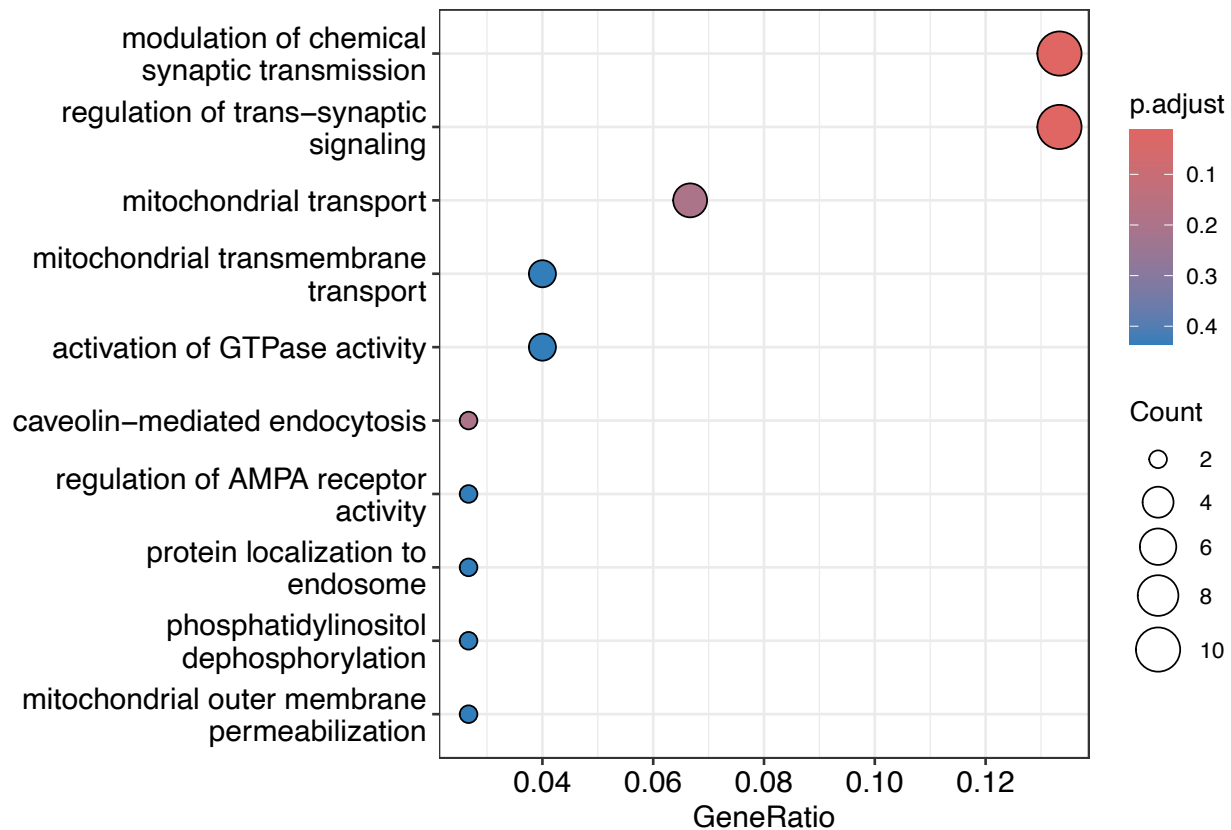


Heatmap of Differentially Expressed Genes



Down regulate

GO Enrichment



UP regulate

GO Enrichment

